

TASK 4: BUILD A VERSION-CONTROLLED DEVOPS PROJECT WITH GIT

Step-by-Step Procedure and Execution Report

Repository: devops-git-project

GitHub: github.com/dhan8292/devops-git-project

1. Objective

The objective of this task is to demonstrate a basic Git and GitHub version-control workflow for a DevOps project. The work includes creating branches, making feature changes, using Pull Requests, maintaining README and .gitignore files, documenting the work in Markdown, and creating a version tag.

2. Tools Used

- Git
- GitHub
- Git Bash
- Markdown

3. Repository and Branching Strategy

Repository: devops-git-project

Working directory: /c/Users/ANUTLA/Desktop/ElvT4/devops-git-project

Branches used:

- main – production/release branch
- dev – development and integration branch
- feature/new-feature – feature development branch
- feature/project-documentation – an additional branch created earlier and retained

Planned workflow:

main → dev → feature/new-feature → Pull Request → dev → Pull Request → main → v1.0.0

Step 1 – Initialize the Git project

The project repository was initialized with Git and the initial project files were prepared.

Command / Action:

README.md

.gitignore

Output / Result:

Task 4 – Version-Controlled DevOps Project with Git

The initial project content was committed with the message: chore: initialize DevOps project.

Step 2 – Create the development branch

A development branch named dev was created from main.

Command / Action:

```
git switch -c dev
```

Output / Result:

The dev branch was created for development and integration work.

Step 3 – Create the feature branch

A feature branch was created for the new documentation change.

Command / Action:

```
git switch -c feature/new-feature
```

Output / Result:

The feature work was isolated from the dev and main branches.

Step 4 – Connect the local repository to GitHub

The local repository was connected to the GitHub repository.

Command / Action:

```
git remote add origin https://github.com/dhan8292/devops-git-project.git
```

Output / Result:

The GitHub repository became the remote named origin.

Step 5 – Synchronize main with GitHub

The GitHub repository already contained an initial README commit. The local main branch was synchronized with the remote using an unrelated-history merge.

Command / Action:

```
git pull origin main --allow-unrelated-histories
```

Output / Result:

A README add/add conflict occurred. The correct local project README was retained, staged, and committed with: merge: sync local main with GitHub.

Step 6 – Push main, dev, and feature branches

The branches were pushed to GitHub and set to track their corresponding remote branches.

Command / Action:

```
git push -u origin main  
git push -u origin dev  
git push -u origin feature/new-feature
```

Output / Result:

All three branches were successfully available on GitHub.

Step 7 – Create feature documentation

On feature/new-feature, the documentation directory and feature documentation file were created.

Command / Action:

```
mkdir -p docs  
touch docs/feature-documentation.md
```

Output / Result:

The file documented the purpose of the feature branch and the feature/new-feature → dev workflow.

Step 8 – Check the new file

Git status was used to confirm that the documentation was untracked before adding it.

Command / Action:

```
git status
```

Output / Result:

Output showed docs/ as an untracked directory.

Step 9 – Stage the feature documentation

The new Markdown file was staged for commit.

Command / Action:

```
git add docs/feature-documentation.md  
git status
```

Output / Result:

Git reported docs/feature-documentation.md under Changes to be committed.

Step 10 – Commit the feature

The documentation was committed on feature/new-feature.

Command / Action:

```
git commit -m "docs: add feature branch documentation"
```

Output / Result:

Commit created successfully: e764b27 docs: add feature branch documentation. The commit contained 18 inserted lines.

Step 11 – Push the feature branch

The feature branch commit was pushed to GitHub.

Command / Action:

```
git push
```

Output / Result:

GitHub received the feature/new-feature branch and the new commit.

Step 12 – Create Pull Request: feature/new-feature → dev

A GitHub Pull Request was created with dev as the base branch and feature/new-feature as the compare branch.

Command / Action:

GitHub Pull Request #1

Output / Result:

The Pull Request was ready to merge and GitHub reported no conflicts.

Step 13 – Merge Pull Request into dev

Pull Request #1 was merged on GitHub.

Command / Action:

GitHub: Merge pull request → Confirm merge

Output / Result:

GitHub reported: Pull request successfully merged and closed.

Step 14 – Update local dev

The local dev branch was switched to and updated from GitHub.

Command / Action:

```
git switch dev
```

```
git pull origin dev
```

Output / Result:

The merge was received as a fast-forward. docs/feature-documentation.md was added with 18 lines.

Step 15 – Verify dev

The development branch was checked after the merge.

Command / Action:

```
git status
```

Output / Result:

Output: On branch dev; Your branch is up to date with origin/dev; nothing to commit, working tree clean.

Step 16 – Create Pull Request: dev → main

A second GitHub Pull Request was created to promote the completed development changes into main.

Command / Action:

GitHub Pull Request: dev → main

Output / Result:

The Pull Request was created successfully with main as the base branch and dev as the compare branch.

Step 17 – Merge dev into main

The dev → main Pull Request was merged on GitHub.

Command / Action:

GitHub: Merge pull request → Confirm merge

Output / Result:

GitHub reported: Pull request successfully merged and closed.

Step 18 – Update local main

The local main branch was switched to and synchronized with GitHub.

Command / Action:

```
git switch main
```

Task 4 – Version-Controlled DevOps Project with Git

```
git pull origin main
```

Output / Result:

The update was a fast-forward. docs/feature-documentation.md was added to main.

Step 19 – Create release tag

An annotated Git tag was created for the completed version.

Command / Action:

```
git tag -a v1.0.0 -m "Release version 1.0.0"
```

Output / Result:

The tag v1.0.0 was created successfully.

Step 20 – Push the release tag

The release tag was pushed to GitHub.

Command / Action:

```
git push origin v1.0.0
```

Output / Result:

GitHub confirmed: [new tag] v1.0.0 -> v1.0.0.

Step 21 – Final working-tree verification

The final main branch status was checked.

Command / Action:

```
git status
```

Output / Result:

Output: On branch main; Your branch is up to date with origin/main; nothing to commit, working tree clean.

Step 22 – Verify branches

All local and remote branches were listed.

Command / Action:

```
git branch -a
```

Output / Result:

Task 4 – Version-Controlled DevOps Project with Git

Verified local branches: dev, feature/new-feature, feature/project-documentation, main. Verified remote branches: origin/dev, origin/feature/new-feature, origin/main.

Step 23 – Verify release tag

The Git tags were listed.

Command / Action:

```
git tag
```

Output / Result:

Output: v1.0.0.

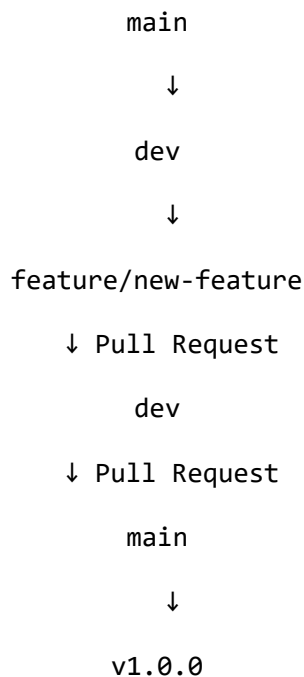
4. Feature Documentation Content

The feature documentation created in docs/feature-documentation.md explains the purpose of the feature branch and records the workflow used to move the change through a Pull Request into dev.

5. Pull Request Workflow Completed

Pull Request	Purpose	Status
feature/new-feature → dev	Integrate feature documentation	Merged
dev → main	Promote completed development changes	Merged

6. Final Git Workflow



7. Final Verification

- ✓ README.md added and maintained.
- ✓ .gitignore used in the repository.
- ✓ dev branch created.
- ✓ feature/new-feature branch created.
- ✓ Feature work committed with a meaningful commit message.
- ✓ Feature branch pushed to GitHub.
- ✓ feature/new-feature → dev Pull Request completed.
- ✓ dev → main Pull Request completed.
- ✓ Markdown documentation added.
- ✓ v1.0.0 annotated tag created.
- ✓ v1.0.0 tag pushed to GitHub.
- ✓ main branch synchronized with origin/main.
- ✓ Final working tree clean.

8. Final Outcome

Task 4 was completed successfully. The project demonstrates a version-controlled DevOps workflow using Git and GitHub, including branching, feature isolation, Pull Requests, documentation, .gitignore usage, and release tagging.

9. Repository

GitHub Repository: <https://github.com/dhan8292/devops-git-project>

END OF TASK 4 DOCUMENTATION